METIS DATA SCIENCE · PROJECT 03

Stack Overflow Tag Prediction

A multi-label NLP classifier that predicts a question's tag set from its title alone — trained on a 400 GB MS SQL Server backup, evaluated across 100 tags.

Garreth Cline · Metis Data Science Bootcamp

25,352 questions · 100 tags · Jaccard = 47.76 · Hamming = 1.02 · 18.6× baseline

TL;DR

A 400 GB database, one NLP pipeline, 100 predicted tags.

THE QUESTION

Can we predict which of 100 common tags belong to a Stack Overflow question using only the title?

THE DATA

400 GB MS SQL dump → 1M rows → filtered to 25,352 high-quality questions; 82,516 tag usages, 6,884 unique.

THE MODEL

TF-IDF titles → 12-classifier benchmark → LinearSVC + GridSearchCV (best C=1).

THE RESULT

Jaccard = 47.76, Hamming = 1.02 — an 18.6× lift over the 2.57 dummy baseline. ~90 of 100 tags at F1 > 0.98.

PassiveAggressive posted a higher raw Jaccard (48.50) — but LinearSVC had the best Hamming loss and stability, so it was the model carried forward. Selection isn't just "max one metric."

From 400 GB to 25,352 rows

Server-side filtering before Python ever sees the data.

RAW DATABASE	SQL EXPORT	SCORE > 50	VIEWS > 10K	TOP-100 TAGS
400 GB	**1,000,000**	**28,183**	**27,666**	**25,352**
MS SQL Server (Brent Ozar dump)	Posts with Title	Community-validated	Popular questions	Final training set

Server-side cleanup

T-SQL strips commas, HTML brackets, nulls before export — saves pulling 400 GB into pandas.

Memory engineering

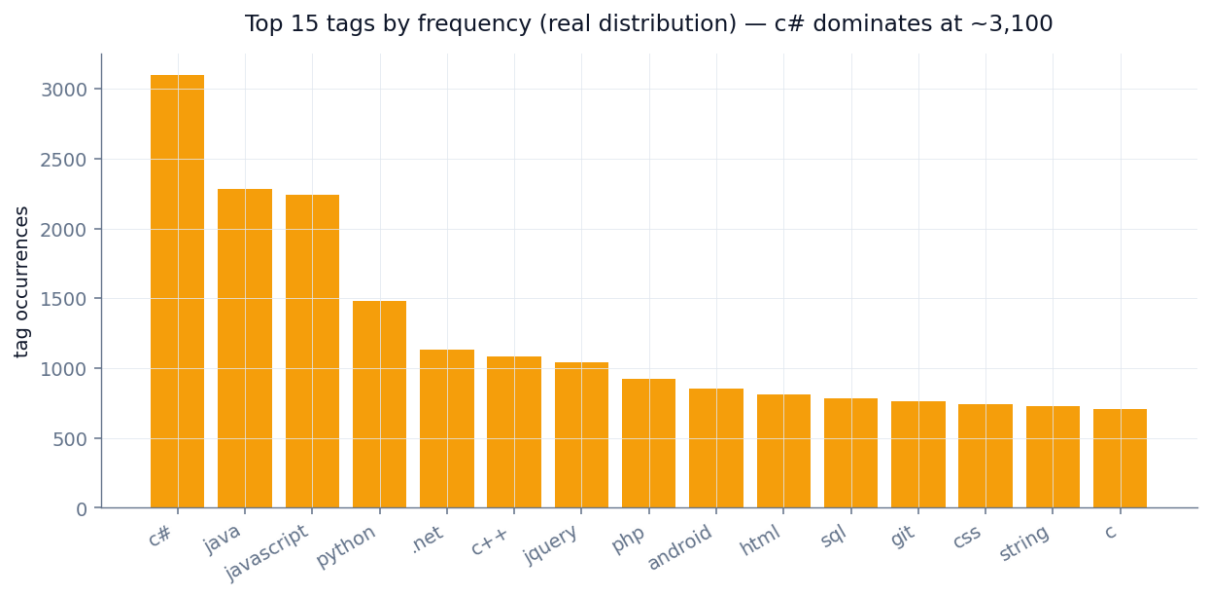
Split peaked at 132 GB RAM; pickle caching + OS paging file to handle it.

Parallelism

n_jobs=32 across the workstation; 100-fold GridSearchCV in minutes.

Tag distribution is a brutal long tail

c# alone appears ~3,100 times; the top 100 cover ~82% of all usages.



Why top-100?

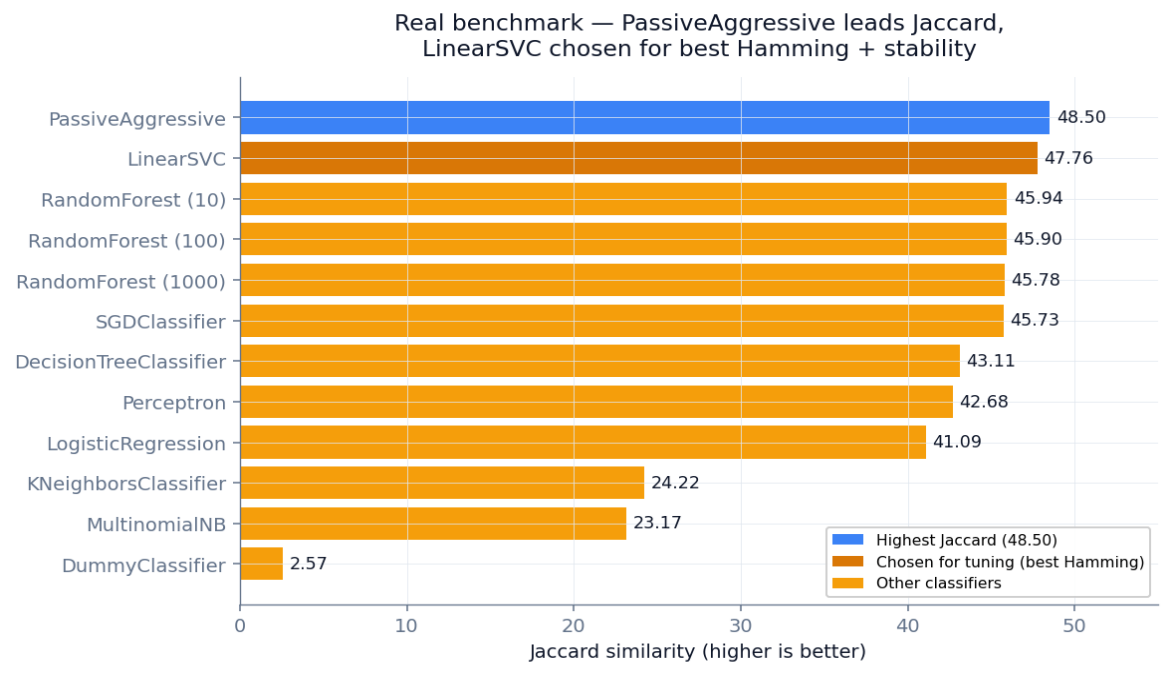
82,516 total tag usages across 6,884 unique tags — but the curve decays fast.

Beyond the top 100, tags have too few examples to learn reliably.

Top-100 captures ~82% of usages while keeping every label well-populated.

12 classifiers — the real ranking

PassiveAggressive leads Jaccard; LinearSVC second and far steadier.



Three findings

Linear models lead — PassiveAggressive (48.50), LinearSVC (47.76), SGD (45.73). Right inductive bias for TF-IDF.

More trees did nothing — all three Random Forests cluster at ~45.8. No interaction structure for an ensemble.

KNN (24.22) and Naive Bayes (23.17) lag — distance breaks down in sparse space; independence assumption fails.

Why LinearSVC, not the higher-Jaccard model?

Reading two metrics at once — the honest selection story.

The tradeoff

PassiveAggressive wins raw Jaccard (48.50) but pays in Hamming loss (1.19).

LinearSVC sits in the best corner — top-tier on BOTH axes (Jaccard 47.76, Hamming 1.02).

Stability: PassiveAggressive's online-update rule makes it erratic run-to-run. LinearSVC is reproducible.

Trading 0.74 Jaccard points for a materially better Hamming loss and reproducibility is the defensible call.

Tuned LinearSVC on held-out test set

GridSearchCV found best C=1; final real numbers below.

JACCARD

47.76

18.6× the dummy baseline (2.57)

HAMMING LOSS

1.02

~1% of the 100-tag matrix mislabeled

F1 > 0.98

~90 of 100

Tags essentially solved

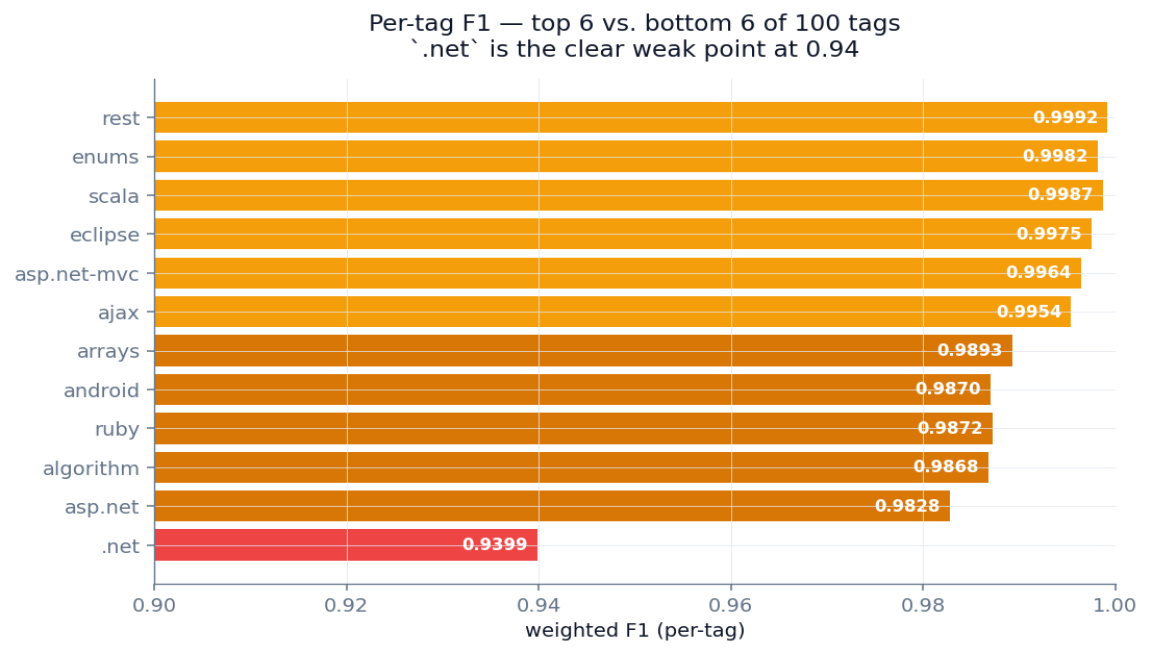
WEAKEST TAG

0.94

`.net` — 25.5% positive-class recall

~90 of 100 tags essentially solved

Real per-tag weighted F1 — top 6 vs. bottom 6.



The spread

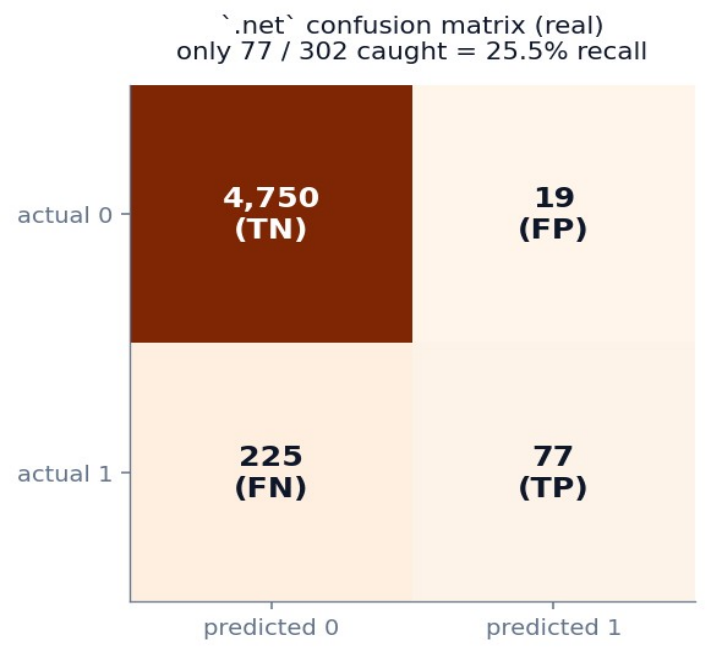
Strongest: rest (0.9992), scala (0.9987), enums (0.9982) — narrow, unambiguous tags.

These tags have distinctive vocabulary that rarely overlaps with others.

The clear outlier: `.net` at 0.94 — the only tag below 0.98.

Why `.net` is the hardest tag

The real confusion matrix exposes a co-occurrence problem.



The problem

Of 302 actual `.net` questions, only 77 were caught — 25.5% positive-class recall.

Its weighted F1 of 0.94 is inflated by 4,750 true negatives — the honest number is the recall.

Why? `.net` almost never appears alone in a title. It co-occurs with c#, vb.net, asp.net, wpf, linq, wcf — and its own top features ARE linq, wcf, assembly, nullable. The binary classifier can't disambiguate on context the OvR wrapper hides.

Fix: hierarchical labels — predict language family first, then `.net` as an implied parent.

The model rediscovered each tag's ecosystem

Real top title words per tag, from the SVC coefficients.

.net

linq, delegate, nullable, wcf, forms, assembly, .net, net

android

device, layout, emulator, listview, intent, textview, webview, activity, android

algorithm

detecting, finding, area, number, graph, binary, sort, tree, algorithm

ajax

underscore, fetch, json, request, google, status, prompt, ajax

asp.net

ip, datatable, request, onclick, master, session, webconfig, asp.net, aspnet

asp.net-mvc

item, render, repository, fire, aspnet, action, controller, partial, mvc

Each tag's learned vocabulary matches its real ecosystem — a sanity check that the model captured genuine patterns, not training-set quirks.

Three ways this gets used

All viable at Jaccard 47.76, Hamming 1.02.

1

Auto-tagging imported archives

Migrating closed forum threads to a tagged knowledge base. Production-ready for the ~90 tags above F1 0.98; human-in-the-loop for .net / c# / c++.

2

Tag-suggestion UI

Autosuggest widget firing on each keystroke as a user types a question title. Top-k predictions become clickable chips.

3

Moderation signal

Flag questions whose author-assigned tags disagree with the model. The model learned canonical patterns from 25K high-quality examples.

What's missing, and where V2 goes

Honest limitations and prioritized future work.

LIMITATIONS

- Title-only input — bodies (error messages, library names) would disambiguate the co-occurring tags.
- Top-100 tags = ~82% of usages, not 100% — the 6,784-tag long tail is out of scope.
- 2010 snapshot — kotlin, rust, tensorflow, react weren't yet prominent.
- Weighted F1 flatters rare tags — per-positive-class recall (the .net 25.5%) is the honest lens.

FUTURE WORK

- Add post Body via BeautifulSoup — biggest gains on co-occurring tags.
- Swap TF-IDF for CodeBERT / StarCoder embeddings — code-aware.
- Hierarchical labels — language family first, then fine-grained tags.
- Per-tag decision-threshold tuning — would lift .net recall directly.

The bottom line

A linear model on TF-IDF titles predicts 100 Stack Overflow tags at Jaccard 47.76 / Hamming 1.02 — ~90 of 100 tags solved, on a fraction of the compute a transformer would need.

What this demonstrates

Data engineering	400 GB MS SQL Server backup; T-SQL export automation; 132 GB-peak memory management.
Model judgment	Chose LinearSVC over a higher-Jaccard model by reading Hamming + stability together.
Honest analysis	Surfaced the .net weak spot (25.5% recall) and designed a hierarchical fix — not just "more data."

STACK Python · pandas · scikit-learn · NLTK · pyarrow · Dask · MS SQL Server · T-SQL · matplotlib